IMPLEMENTATION NOTE • MHD-WAAM THESIS • EDIT 4

How the Droplets Were Added

Where the code goes, why it goes there, and what each term means

laserbeamFoam, OpenFOAM ESI v2506 | four files touched, one created | reversible

THE SHORT VERSION

A droplet is **placed**, not injected through a boundary. Once every 33 ms the solver finds the free surface under the torch, and in every cell inside a 1.105 mm sphere just above it, sets `alpha.metal = 1`, `T = 1506 K` and `epsilon1 = 1`. That is a ball of liquid aluminium appearing in mid-air above the pool.

Its **momentum** cannot be placed the same way, because the solver computes velocity rather than storing it. So the impact is a **body force** applied to that same metal for 2.46 ms, sized so its impulse equals exactly $m_{drop} \times v_{impact}$.

It happens **once per timestep, outside the pressure-velocity loop**. That placement is not stylistic — inside the loop the injection would be silently erased, for a reason given in section 6.

Nothing else in the solver changed. A case without droplet entries in its dictionary runs exactly as it did before, verified to six columns and five significant figures.

1. Why droplets are in this model at all

The first arc runs melted almost nothing, and chasing that produced the finding this edit exists to answer. On a properly sized domain, with the boundary conditions actually being applied, **the arc alone saturates at about 832 K**. Al–5%Mg has a solidus of 815 K and a liquidus of 906 K, so the arc gets metal 17 K past the point where it stops being fully solid and stops 74 K short of the point where it becomes fully liquid. A mushy skin of about 1 mm^3, and no weld pool at any time.

That is not a defect. It is how GMAW works. The workpiece is not melted by the arc — it is melted by **superheated filler droplets**, and the arc's job is mostly to melt the wire. Zhao's assumption 1 models this as liquid metal entering at 1506 K.

The energy those droplets carry

This is worth doing by hand, because it is the whole justification and it needs no simulation. The wire is 1.2 mm ER5356 fed at 0.15 m/s, which is Zhao Table 2's 900 cm/min:

```
volume rate = pi*(0.6e-3)^2 * 0.15 = 1.696e-7 m^3/s
mass rate = x 2650 kg/m^3 = 4.495e-4 kg/s = 0.450 g/s

enthalpy per kg, from 300 K to liquid at 1506 K
  c_p(815 - 300) 940 x 515 = 484,100 heat the solid to the solidus
  L_f = 358,000 melt it
  c_p(906 - 815) 940 x 91 = 85,540 cross the mushy range
  c_p(1506 - 906) 940 x 600 = 564,000 superheat to the wire tip
                                 ---------
                                 1.492 MJ/kg
```

```
droplet power = 4.495e-4 x 1.4916e6 = 671 W
arc power = 1512 W
droplets add 44%
```

> **AND THEY DO NOT MERELY ADD POWER**
>
> 1506 K is **600 K above the liquidus**. That metal arrives already molten: its energy needs no conduction to become pool, it *is* pool the instant it lands.
>
> This is why the arc-alone result understates the gap so badly. The arc must conduct its heat into cold solid aluminium and fight the plate's own conductivity the whole way. The droplets simply deliver liquid.

2. Background: what the solver is actually tracking

To follow where the code goes, three things about the solver have to be clear first.

2.1 alpha.metal, the phase fraction

This is a **Volume of Fluid** solver. It does not track a surface. Every cell carries a number `alpha.metal` between 0 and 1: the fraction of that cell's volume occupied by metal rather than argon. A cell full of aluminium holds 1, a cell full of shielding gas holds 0, and the free surface is wherever cells hold something in between. Density, viscosity and conductivity are blended cell by cell using it.

So **putting metal somewhere means writing 1 into alpha.metal there**. There is no other step. That is the whole mechanism by which a droplet comes into existence in this model.

2.2 epsilon1, the liquid fraction

Separately from which *phase* a cell holds, metal cells carry `epsilon1`: how much of that metal is liquid rather than solid. It is computed from temperature across the mushy range,

$$\texttt{epsilon1} = \max\left(\min\left(\frac{T - T_{solidus}}{T_{liquidus} - T_{solidus}}, 1\right), 0\right)$$

and it matters because of **Darcy damping**. A finite-volume solver has no concept of a solid; left alone it would let solid aluminium flow like a fluid. The enthalpy-porosity method prevents that by treating the mushy zone as a porous medium and adding a momentum sink whose strength rises without limit as the liquid fraction falls:

$$\texttt{DC} = K \cdot \frac{(1 - \texttt{epsilon1})^2}{\texttt{epsilon1}^3 + \texttt{small}}$$

At `epsilon1 = 1` this is negligible and the metal flows freely. At `epsilon1 = 0` it is enormous and the metal is frozen in place. **This is the reason section 7.4 exists.**

2.3 The PIMPLE loop

Incompressible flow has no equation of state linking pressure to density, so pressure cannot be computed directly — it has to be found by demanding that the velocity field end up divergence-free. That produces a chicken-and-egg problem: velocity depends on pressure, pressure is determined by velocity. PIMPLE resolves it by iterating. Each timestep runs several **outer correctors**, and within each, several **pressure correctors**, until the two are consistent.

The consequence that matters here: **everything inside that loop runs more than once per timestep.** Anything placed there must be safe to execute two or three times, and anything that should happen exactly once must go outside it.

3. The solver's time loop, as it stands

This is `laserbeamFoam.C` reduced to its structure, with the one added line marked. Everything else was already there.

```
while (runTime.run())
{
    #include "readControls.H"          Courant numbers, timestep
    ++runTime;
    Info<< "Time = " ...

    #include "updateDropletSource.H"   <-- ADDED. Once per timestep.

    while (pimple.loop())             outer correctors, 2-3 per step
    {
        #include "alphaEqnSubCycle.H"   advect the interface
        #include "updateProps.H"        blend rho, mu, kappa by alpha
        laser.updateDeposition(...)     zero power in our cases
        mixture.correct();

        #include "UEqn.H"               momentum  <-- one term added
        mthd->solve(phi, U);            magnetic, off in our cases
        #include "updateArcSource.H"    the Goldak ellipsoid
        #include "TEqn.H"               energy, with latent heat

        while (pimple.correct())
            #include "pEqn.H"           pressure correctors
    }
    runTime.write();
}
```

The injection sits between the time increment and the PIMPLE loop. Section 6 explains why it cannot sit inside it.

4. Three ways to add a droplet, and why one was chosen

APPROACH	WHAT IT IS	WHY NOT / WHY YES
A — boundary inlet	Zhao's method. The top face is a velocity inlet; inside the wire radius it injects liquid at 1506 K and 0.15 m/s, and droplets form, neck and detach as part of the solution	**Rejected for now.** Needs the wire tip and the necking resolved. Zhao uses 0.25 mm cells and still calls droplet transfer the expensive part. At 0.5 mm the wire cross-section spans 2.3 inlet faces and jitters between 2 and 4 as the torch crosses the grid
B — smooth volumetric source	Add mass and enthalpy as continuous source terms spread over a disc under the torch, with a separate m_dot*v force for the momentum	**Kept as the fallback.** Robust at any resolution and gets the bead volume and thermal field right, but the metal appears as a smear rather than an object, so there is no geometric displacement and no crater

APPROACH	WHAT IT IS	WHY NOT / WHY YES
C — injected sphere	Place a ball of liquid above the pool at the droplet's real radius and let the existing VOF machinery handle the fall, the impact and the coalescence	**Chosen.** The droplet is an object, so it displaces pool metal geometrically. Follows Zhou [17] and Bai [18], both cited in Zhao's introduction. Detachment is imposed from Zhao Eq. 30 rather than predicted, which is the trade being made

Zhao criticises exactly the method chosen, writing that a fixed sphere generated at a fixed frequency "was inconsistent with the process of droplet formation, growth, detachment, and entry into the molten pool". That objection is about not *predicting* detachment, and it is correct. The answer is that radius and frequency are not invented here — they come from Zhao's own Eq. 30 and are validated in that paper against a simulated free-flying droplet, 1.105 mm theoretical against 1.114 mm simulated, 0.88% error. The criticism should be quoted in the thesis and answered, not avoided.

5. Where the code actually goes

Four touch points. One new file and three single-anchor edits, each verified to match exactly once before anything is written.

FILE	CHANGE
`createFields.H`	Reads the droplet parameters from `constant/caseParameters` and declares the `dropletForce` field. Follows the arc's own convention: the PRESENCE of a key switches the feature on, so a case without droplet entries is untouched
`laserbeamFoam.C`	One `#include`, placed between the time increment and the PIMPLE loop
`updateDropletSource.H`	**NEW.** Decides whether a droplet is due, finds the free surface, places the sphere, and sets the momentum source
`UEqn.H`	One term added to the momentum equation, beside the two body forces already there

5.1 The momentum equation, before and after

The change is one line, and its form is copied from what was already there rather than invented:

```
fvVectorMatrix UEqn
(
    fvm::ddt(rho, U) + fvm::div(rhoPhi, U)
  + MRF.DDt(rho, U)
  + turbulence->divDevRhoReff(rho, U)
  + fvm::Sp(DC, U)              <- Darcy: the solid-metal momentum sink
  - Marangoni*damper            <- surface-tension gradient force
  - JcrossB*damper              <- Lorentz force, zero in these runs
  - dropletForce*damper         <- ADDED
 ==
    fvOptions(rho, U)
);
```

*damper = 2*rho/(rho1+rho2) is the density weighting the existing surface forces already use. It is Zhao's own 2*rho/(rho_metal+rho_gas) from Eqs 14-15, and the droplet force inherits it for free by being written the same way.*

6. Why the injection is outside the PIMPLE loop

This was the single most important thing reading the source revealed, and it is invisible from outside. Inside `alphaEqnSubCycle.H`:

```
if (pimple.nCorrPIMPLE() > 1)
{
    if (pimple.firstIter())
    {
        alpha1.storePrevIter();      // remember alpha at step start
        phi.storePrevIter();
        U.storePrevIter();
    }
    else
    {
        alpha1 = alpha1.prevIter();  // <-- RESET, every later iteration
        U   = 0.5*U.prevIter()   + 0.5*U;
        phi = 0.5*phi.prevIter() + 0.5*phi;
    }
}
```

The reason this exists is sound: on outer iterations after the first, the interface should be re-advected from its state at the *start* of the timestep using an improved estimate of the flux, not advected again on top of an already-advected field. So alpha is rewound each time.

THE CONSEQUENCE

Anything written into `alpha1` from inside the PIMPLE loop is **discarded on the second and third outer iterations**. A droplet injected there would be placed, erased, placed again, erased again, and whether any of it survived would depend on which iteration happened to be last.

There is **no error and no warning**. The run completes. The droplets would simply be smaller than asked for, by a factor nobody could predict from the dictionary.

Placing the injection before `while (pimple.loop())` means `alpha1.storePrevIter()` captures the state *including* the new droplet, and every outer iteration rewinds to that. The droplet is placed exactly once and survives.

7. The four things a droplet has to bring

A droplet is not one quantity. It is mass, energy, momentum and phase state, and each enters the solver by a different route.

7.1 Mass — written directly into alpha

Because `alpha.metal` *is* the presence of metal, placing a droplet is writing 1 into every cell whose centre lies inside the sphere. No source term, no equation — the metal is simply there on the next timestep, and the VOF scheme advects it like any other metal.

7.2 Energy — written directly into T

The same cells are set to 1506 K. Note what is **not** needed: no latent heat has to be supplied, because the metal is arriving already liquid. The 1.492 MJ/kg computed in section 1 is not added as a heat source anywhere — it is implicit in the fact that 2650 kg/m^3 of aluminium now exists at 1506 K where before there was argon at 300 K. The energy equation inherits it automatically.

7.3 Momentum — a body force, because velocity is solved for

Mass and energy can be placed because `alpha1` and `T` are *stored* and then transported. Velocity is different: `UEqn` **solves** for U every outer iteration, so a value written into U beforehand is only an initial guess and is overwritten within microseconds of simulated time. Placing a droplet at 0.9 m/s by assignment does nothing at all.

Momentum therefore has to enter the way every other force does — as a term in the equation. The droplet's momentum is m_{drop} x v_{impact}, which is an impulse; spread over the impact duration it is a force:

```
impact duration tau = d / v = 2 x 1.105e-3 / 0.9 = 2.456 ms
      (the time to pass one droplet diameter into the pool)

F = m_drop * v / tau = 1.498e-5 * 0.9 / 2.456e-3 = 5.49e-3 N during impact
F = m_dot * v = 4.495e-4 * 0.9 = 4.05e-4 N time-averaged

duty cycle = tau * f = 2.456e-3 * 30 = 7.37%, so peak/mean = 13.6
```

The force is distributed over the metal in the sphere, weighted by `alpha*V` so it acts on liquid and not on argon, and its centre translates downward at 0.9 m/s so it follows the droplet rather than staying where the droplet was born — over 2.46 ms the droplet travels a full diameter.

7.4 Phase state — epsilon1, the trap

> **WITHOUT THIS LINE, THE DROPLETS DO NOT MOVE**
>
> `UEqn` computes the Darcy damping `DC` from `epsilon1`, and it runs **before** `TEqn` recomputes `epsilon1` from the new temperature.
>
> So on the timestep a droplet is injected, the cells hold metal at 1506 K — unambiguously liquid — but `epsilon1` is still whatever it was when those cells were argon, which is 0. Darcy sees a fully solid cell and applies a momentum sink large enough to pin it in place.
>
> The droplet would land and freeze on contact, and the symptom would be "the droplets are not moving" with nothing in any output to explain it. Setting `epsilon1 = 1` alongside `alpha1` and `T` costs one line and avoids an evening.

8. Making the mass exact

A 2.21 mm sphere on a 0.5 mm mesh spans 4.4 cells. How much volume it actually covers depends on where its centre falls relative to the grid — a sphere centred on a cell corner covers a different set of cells than one centred on a cell centre. As the torch travels, that alignment changes continuously.

Firing droplets on a **clock** — one every 33.3 ms — would therefore deliver a mass flow that jitters by several percent with no physical cause. Instead the solver accumulates the volume it owes:

```
dropOwed_ += dropVdot_*dt;              // owed volume grows every step

if (dropOwed_ >= dropVdrop_)            // enough for a whole droplet?
{
    ... place the sphere, summing the volume ACTUALLY added ...
    placed += (1.0 - a1[celli])*Vc[celli];   // only the argon fraction

    dropOwed_ -= placed;                // the shortfall carries forward
}
```

Only the argon fraction of each cell counts as new volume. Counting the whole cell would over-report whenever a sphere clips metal that was already there.

Because the error carries forward rather than accumulating, **the time-averaged mass flow is exactly 0.450 g/s** however the cells happen to fall. Individual droplets vary; the total does not. The solver prints both figures per droplet so the drift is visible rather than assumed:

```
Droplet 1 at t=0.0334  centre (0.668 0 1.605) mm  22 cells
          placed 5.61 mm^3 of 5.652 nominal  (carried 0.042 mm^3)
```

9. Where the droplet is placed

Two coordinates come from the arc and one is found by looking.

x and y come from the arc's own variables, not from a separate calculation: `xc = arc_x0_ + arcV_*tNow`, the identical expression `updateArcSource.H` uses for the Goldak ellipsoid. If the wire and the arc ever disagreed about where the torch is, the metal would land away from the hot spot and every result downstream would be quietly wrong. One source of truth is the only safe arrangement.

z is found by searching. The solver scans for the highest metal within one droplet radius of the torch axis — which is the free surface, wherever the bead has grown to — and places the sphere centre one standoff plus one radius above it. The search is reduced across processors, because that column of cells can straddle a decomposition boundary.

WHY THE STANDOFF IS NOT COSMETIC

The sphere must be clear of existing metal. If it overlapped the pool, the injection would set `alpha = 1` in cells that already held metal, and the volume accounting would credit a droplet that was largely already there.

The `(1 - alpha)` weighting in section 8 protects the accounting, but a sphere half-buried in the pool is still not a droplet arriving. One cell of clearance, 0.5 mm, costs 0.55 ms of flight at 0.9 m/s and gravity adds 0.005 m/s over that — negligible physically, and it keeps the injection clean.

10. What this does not model

- **Droplet formation.** The pendant drop growing at the wire tip, necking under the electromagnetic pinch, and detaching are not simulated. Radius and frequency are imposed from Zhao Eq. 30.
- **The flight.** The sphere is created just above the pool rather than at the wire tip, so plasma drag and the electromagnetic force during flight play no part. Their effect is contained in the single number v_{impact} = 0.9 m/s.
- **Spatter.** Nothing is ejected. At a Weber number of rho*v^2*d/sigma = 5.6 the droplet coalesces gently rather than splashing, so this is a small omission — splashing needs We in the hundreds.
- **The droplet's internal circulation**, which arrives as a uniform 1506 K and a uniform downward force rather than as a circulating blob.
- **Resolution.** 4.4 cells across a diameter is thin. The usual guidance for holding a VOF droplet together is 8 to 10. Whether it is enough is a measurement, not an argument — see section 11.

11. How to tell whether it is working

CHECK	WHAT IT TESTS	PASS
Startup banner	Arithmetic, before any physics	0.4495 g/s, 30 Hz, 5.652 mm^3, 4.05e-4 N — all four derivable by hand
`cells` per droplet	Mesh resolution	~20. Below 4 the solver warns outright
`placed` vs `nominal`	Mass conservation	Individually within a few percent, and `carried` not growing
Anything above 906 K	The point of the whole edit	Liquid exists. The arc alone never achieved this
Bead cross-section	The inlet as a whole	Near **8.5 mm^2** — fixed by mass conservation before any simulation, so a miss means the inlet is wrong
Mirror symmetry about y = 0	Numerical error, free on a full domain	Any asymmetry at B = 0 is accumulated round-off, and its growth is the noise floor as a time series
A case with no droplet keys	That the patch is inert when unused	**Verified:** six columns identical to five significant figures against the same case on the unpatched solver

The last row is the one that makes everything else meaningful. Because a case without `droplet_radius_m` in its dictionary produces bit-comparable output on the patched solver, any difference seen when droplets are switched on is attributable to the droplets and not to the patch. Without that check, every droplet result would have two possible explanations.